

Venus Relative Position Manager

Security Assessment

CertiK Assessed on Mar 13th, 2026





Certik Assessed on Mar 13th, 2026

Venus Relative Position Manager

The security assessment was prepared by Certik.

Executive Summary

TYPES

Lending

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 02/23/2026

Final report published on 03/13/2026

Vulnerability Summary



21

Total Findings

18

Resolved

2

Timelock

0

Partially Resolved

1

Acknowledged

0

Declined

2 Centralization

2 Timelock

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

3 Medium

3 Resolved

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

3 Minor

3 Resolved

Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

13 Informational

12 Resolved, 1 Acknowledged

Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | VENUS RELATIVE POSITION MANAGER

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Overview

[PR-47](#)

Dependencies

[Third Party Dependencies](#)

[Recommendations](#)

Findings

[VRP-05 : Centralization Related Risks](#)

[VRP-06 : Centralized Control Of Contract Upgrade](#)

[VRP-07 : Profit Conversion Path Does Not Account For Redeem Fee](#)

[VRP-19 : Zero Division In Utilization When DSA Collateral Factor Is Zero](#)

[VRP-21 : ``closeWithLoss\(\)`` Can Increase Position Leverage Beyond RPM Policy Target Via Non-Proportional DSA Redeem](#)

[VRP-08 : Conservative Rounding Is Not Applied In ``\_getUtilizationInfo\(\)``](#)

[VRP-09 : ``getPositionAccountAddress\(\)`` Can Return Incorrect Address After Implementation Upgrade](#)

[VRP-10 : ``effectiveLeverage`` Is Fixed At Activation And Can Exceed New Max After Collateral Factor Updates](#)

[VRP-01 : Discussion On ``POSITION\_ACCOUNT\_IMPLEMENTATION``](#)

[VRP-02 : Discussion On Dust Transfer Reverts For Tokens With Blacklists](#)

[VRP-03 : Discussion On Hardcoded 2% Full-Close Repay Bump](#)

[VRP-04 : Discussion On Proportional Close Tolerance](#)

[VRP-11 : Incompatibility With Deflationary Tokens \(Non-Standard ERC20 Token\)](#)

[VRP-12 : Typos And Inconsistencies](#)

[VRP-13 : Dust Amount Supplied Can Mint 0 VToken](#)

[VRP-14 : DSA Market Is Not Exited When Deactivated](#)

[VRP-15 : Potential Incorrect ``amountRedeemed`` Emitted In ``PositionClosed`` Event During ``closeWithLoss\(\)`` When DSA Equals Short](#)

[VRP-16 : Missing Input Validation](#)

VRP-17 : `minAmountOut` Parameters Only Guard Slippage Not Total Repayment Requirements

VRP-18 : External Transfers Into `PositionAccount` Affect Accounting Semantics

VRP-20 : Missing Emit Events

Appendix

Disclaimer

CODEBASE | VENUS RELATIVE POSITION MANAGER

Repository

<https://github.com/VenusProtocol/venus-periphery/>

Commit

Base: [d60851ea97d62ca76d7837e15edaf10f60b4b68a](#)

Update1: [b7a31dc6380e3f50ba744dbb97473e108c973b7e](#)

Update2: [29f35a0de3e29315ff0bc167e97f05972215ab91](#)

Update3: [1ac5da3a62e13ec0a4796f59ed3c092f156516b2](#) Update4: [1f0fb466bb8a39dec82f19ee63ebe202ab9898e6](#)

AUDIT SCOPE | VENUS RELATIVE POSITION MANAGER

VenusProtocol/venus-periphery

 RelativePositionManager.sol

 PositionAccount.sol

 IPositionAccount.sol

 IRelativePositionManager.sol

APPROACH & METHODS | VENUS RELATIVE POSITION MANAGER

This audit was conducted for Venus to evaluate the security and correctness of the smart contracts associated with the Venus Relative Position Manager project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

OVERVIEW | VENUS RELATIVE POSITION MANAGER

This audit concerns the changes made in the in scope files outlined in the following PR:

- [PR-47](#)

Note that any centralization risks present in the existing codebase before these PRs were not considered in this audit and only those added in these PRs are addressed in the audit. We recommend all users carefully review the centralization risks, much of which can be found in our previous audits, which can be found here: <https://skynet.certik.com/projects/venus>.

PR-47

This PR introduces the **RelativePositionManager**, which manages isolated leveraged positions across long/short markets with DSA (Default Settlement Asset) collateral. It adds core logic to activate positions, enforce leverage bounds, open/close positions with profit/loss, and handle principal accounting through deterministic **PositionAccount** clones.

RelativePositionManager

This contract handles position setup (including DSA selection and leverage limits), tracks each position's state, and enforces risk controls like utilization and max borrow. It lets users open or scale positions by borrowing from the short market and swapping into the long asset, with the DSA principal serving as the initial collateral and the long position itself contributing collateral value as it grows. It supports closing with either profit or loss using proportional close logic and tolerance bands. It also deploys deterministic **PositionAccount** contracts, manages principal accounting, and wraps up positions with dust handling and deactivation.

PositionAccount

This contract holds a user's position account for a specific long/short pair. It's a deterministic clone that stores the owner and market pair, and forwards leverage enter/exit calls to the **LeverageStrategiesManager**. It includes a single-asset exit when collateral equals debt and a dust sweep to return leftovers to the owner. All entry points are restricted to the **RelativePositionManager**, which is responsible for safety checks and access control.

DEPENDENCIES | VENUS RELATIVE POSITION MANAGER

Third Party Dependencies

The protocol is serving as the underlying entity to interact with third party protocols. The third parties that the contracts interact with are:

- ERC20 Tokens
- Swap Protocols
- Oracles

The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. Moreover, updates to the state of a project contract that are dependent on the read of the state of external third party contracts may make the project vulnerable to read-only reentrancy. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Recommendations

We recommend constantly monitoring the third parties involved to mitigate any side effects that may occur when unexpected changes are introduced, as well as vetting any third party contracts used to ensure no external calls can be made before updates to its state.

FINDINGS | VENUS RELATIVE POSITION MANAGER



21

Total Findings

0

Critical

2

Centralization

0

Major

3

Medium

3

Minor

13

Informational

This report has been prepared for Venus to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 21 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
VRP-05	Centralization Related Risks	Centralization	Centralization	1h Timelock
VRP-06	Centralized Control Of Contract Upgrade	Centralization	Centralization	1h Timelock
VRP-07	Profit Conversion Path Does Not Account For Redeem Fee	Logical Issue	Medium	Resolved
VRP-19	Zero Division In Utilization When DSA Collateral Factor Is Zero	Logical Issue	Medium	Resolved
VRP-21	<code>closeWithLoss()</code> Can Increase Position Leverage Beyond RPM Policy Target Via Non-Proportional DSA Redeem	Logical Issue	Medium	Resolved
VRP-08	Conservative Rounding Is Not Applied In <code>_getUtilizationInfo()</code>	Logical Issue	Minor	Resolved
VRP-09	<code>getPositionAccountAddress()</code> Can Return Incorrect Address After Implementation Upgrade	Logical Issue	Minor	Resolved
VRP-10	<code>effectiveLeverage</code> Is Fixed At Activation And Can Exceed New Max After Collateral Factor Updates	Logical Issue	Minor	Resolved
VRP-01	Discussion On <code>POSITION_ACCOUNT_IMPLEMENTATION</code>	Design Issue	Informational	Resolved
VRP-02	Discussion On Dust Transfer Reverts For Tokens With Blacklists	Design Issue	Informational	Resolved

ID	Title	Category	Severity	Status
VRP-03	Discussion On Hardcoded 2% Full-Close Repay Bump	Design Issue	Informational	● Acknowledged
VRP-04	Discussion On Proportional Close Tolerance	Logical Issue	Informational	● Resolved
VRP-11	Incompatibility With Deflationary Tokens (Non-Standard ERC20 Token)	Volatile Code	Informational	● Resolved
VRP-12	Typos And Inconsistencies	Inconsistency	Informational	● Resolved
VRP-13	Dust Amount Supplied Can Mint 0 VToken	Logical Issue	Informational	● Resolved
VRP-14	DSA Market Is Not Exited When Deactivated	Logical Issue	Informational	● Resolved
VRP-15	Potential Incorrect <code>amountRedeemed</code> Emitted In <code>PositionClosed</code> Event During <code>closeWithLoss()</code> When DSA Equals Short	Logical Issue	Informational	● Resolved
VRP-16	Missing Input Validation	Logical Issue	Informational	● Resolved
VRP-17	<code>minAmountOut</code> Parameters Only Guard Slippage Not Total Repayment Requirements	Logical Issue	Informational	● Resolved
VRP-18	External Transfers Into <code>PositionAccount</code> Affect Accounting Semantics	Logical Issue	Informational	● Resolved
VRP-20	Missing Emit Events	Coding Style	Informational	● Resolved

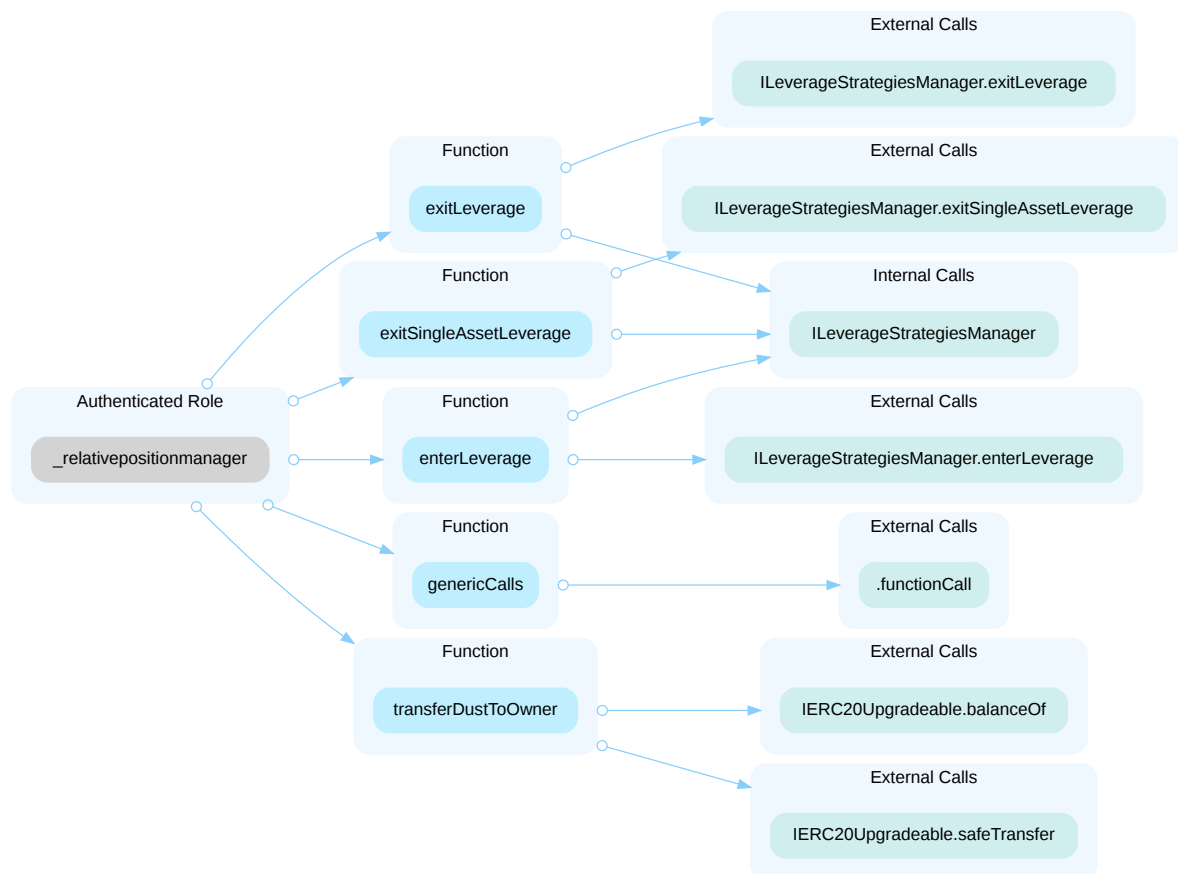
VRP-05 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	● Centralization	PositionAccount.sol (Base): 171, 207, 238, 255, 272	● 1h Timelock

Description

In the contract `PositionAccount`, the role `_relativepositionmanager` has authority over the functions shown in the diagram below. Any compromise to the `_relativepositionmanager` account may allow the hacker to take advantage of this authority and do the following.

- Can open or close leverage on behalf of the position account, impacting collateral and debt.
- Can execute arbitrary external calls from the position account, which could drain all funds.
- Can sweep arbitrary token balances from the position account to the owner.



In the contract `RelativePositionManager`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `pause()`
- `unpause()`
- `setPositionAccountImplementation()`

- `addDSAVToken()`
- `setDSAVTokenActive()`
- `executePositionAccountCall()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Halt or resume all state-changing user operations.
- Change the implementation used for new position account clones.
- Add new DSA vToken markets.
- Enable/disable DSA vTokens for new activations.
- Execute arbitrary calls from any PositionAccount via genericCalls. Potentially draining their funds.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND

- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

I Alleviation

[Venus, 03/03/2026]: Issue acknowledged. I won't make any changes for the current version.

The ownership of all contracts will be transferred to the standard Venus Normal Timelock. ACM permissions will be assigned to specific timelocks based on the required level of control, normal, fast-track, or critical, and a guardian will be designated for critical setters. All permission assignments will be executed through a VIP and approved via Venus governance, accompanied by a community post to ensure transparency to users.

Governance Addresses: <https://docs-v4.venus.io/deployed-contracts/governance#bnb-chain-mainnet>.

VRP-06 | Centralized Control Of Contract Upgrade

Category	Severity	Location	Status
Centralization	● Centralization	RelativePositionManager.sol (Base): 26	● 1h Timelock

Description

The proxy admin has the authority to update the implementation contract `RelativePositionManager`.

Any compromise to the proxy admin account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (2/3, 3/5) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (2/3, 3/5) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

I Alleviation

[Venus, 03/03/2026]: Issue acknowledged. I won't make any changes for the current version.

The admin of the contracts on BNB Chain (where these contracts will be deployed) will be the ProxyAdmin contract at 0x1BB765b741A5f3C2A338369DAb539385534E3343. The owner of this ProxyAdmin is the Normal Timelock contract at 0x939bD8d64c0A9583A7Dcea9933f7b21697ab6396, which is responsible for executing standard Venus Improvement Proposals (VIPs) on BNB Chain. For normal VIPs, the governance process includes: 24 hours of voting 48 hours of timelock delay before execution. Therefore, any upgrade to these contracts can only be executed through a Normal VIP, ensuring that changes go through the Venus community governance process.

VRP-07 | Profit Conversion Path Does Not Account For Redeem Fee

Category	Severity	Location	Status
Logical Issue	● Medium	RelativePositionManager.sol (Base): 882, 887	● Resolved

Description

The function `_redeemLongAndSwapToDSA()` calls `redeemUnderlyingBehalf(positionAccount, amountToRedeem)` and then immediately swaps exactly `amountToRedeem`. If `COMPTROLLER.treasuryPercent() > 0`, the redeem returns less than `amountToRedeem`, so the contract's long underlying balance can be insufficient. The next call to `_performSwap()` transfers `amountToRedeem`, which reverts on insufficient balance.

This prevents users from closing with a profit if the `COMPTROLLER.treasuryPercent() > 0`.

Recommendation

We recommend refactoring the code to account for if `COMPTROLLER.treasuryPercent() > 0`.

Alleviation

[CertiK, 03/03/2026]: The client made the recommended changes resolving this finding in commits

- [f3c32abef92d7539fcbd6d1a189a5c0628c1d11d](#);
- [8d0c9904e849c9fd5fa93232592263028a5605](#).

VRP-19 | Zero Division In Utilization When DSA Collateral Factor Is Zero

Category	Severity	Location	Status
Logical Issue	● Medium	RelativePositionManager.sol (Update1): 1243	● Resolved

Description

The function `_getUtilizationInfo()` reads `dsaCF` from Comptroller and computes:

```
actualCapitalUtilized = excessBorrowUSD > 0 ? ceilDiv(excessBorrowUSD * 1e18, dsaCF) : 0;
```

Note that `ceilDiv()` does not handle division by zero. If the DSA market collateral factor is set to 0, and `excessBorrowUSD > 0`, this path reverts causing a denial of service for those paths that rely on `_getUtilizationInfo()`.

Recommendation

We recommend explicitly handling the case when `dsaCF == 0`.

Alleviation

[CertiK, 03/06/2026]: The client made the recommended changes resolving the finding in commit [a1340bf4e8feefd7481216b13e20b37e2f032a06](#).

VRP-21 | `closeWithLoss()` Can Increase Position Leverage Beyond RPM Policy Target Via Non-Proportional DSA Redeem

Category	Severity	Location	Status
Logical Issue	● Medium	RelativePositionManager.sol (Update2): 409, 508-511, 524, 1021	● Resolved

Description

`closeWithLoss()` enforces proportional-close constraints on first-leg long redemption and debt-repay math, but does not proportionally constrain `dsaAmountToRedeemForSecondSwap` to `closeFractionBps`. The second-leg DSA amount is only checked against total principal, not against BPS-derived close bounds.

As a result, users can remove comparatively more DSA collateral than implied by the selected close fraction, which can increase effective leverage relative to RPM's intended leverage policy (set/enforced during open/scale flows). This shifts enforcement from RPM policy checks to downstream solvency checks in `LeverageStrategiesManager/Comptroller`.

Additionally, effective on-chain redeemed proportionality can drift when treasury gross-up is applied (`ceil(amount / (1 - treasuryPercent)))`). Even if raw inputs appear within proportional bounds, actual redeemed collateral may exceed those bounds unless validation is performed on grossed/effective amounts.

Scenario

- User opens a high-leverage position near RPM's allowed bound.
- User calls `closeWithLoss()` with very small `closeFractionBps` (e.g., 1 bps), making `amountToRepaySecond` small.
- User sets `dsaAmountToRedeemForSecondSwap` to a much larger amount (up to principal cap).
- RPM accepts because second-leg DSA check only validates $\leq$ `principalUnderlying`.
- Second leg redeems/swaps large DSA and repays only the small computed debt amount.

The net effect is a disproportionate collateral reduction vs debt reduction, increasing leverage relative to RPM close-fraction policy. Note that the position can only be leveraged up to the LM/Comptroller account-safety checks.

Recommendation

We recommend either:

- Enforce a `closeFractionBps`-aligned BPS/tolerance cap on `dsaAmountToRedeemForSecondSwap`, and apply the cap to the effective redeemed amount (including treasury gross-up), not just raw user input.
- Enforce a post-close policy leverage invariant (not only solvency) before executing second-leg redemption, using the same treasury-adjusted effective amount model as execution.

For consistency, apply the same effective-amount validation principle to any other close path where treasury gross-up can

increase actual redeemed collateral versus validated raw inputs.

I Alleviation

[Venus, 03/11/2026]: The `dsaAmountToRedeemForSecondSwap` is intentionally unconstrained because `closeWithLoss` handles underwater positions where the loss magnitude is unpredictable. Capping DSA input proportionally to `closeFractionBps` could make losing positions uncloseable if the allowed DSA isn't enough to cover the shortfall. The debt repayment (`amountToRepaySecond`) is already proportionally bounded by BPS.

Secondly, the proportional check on redeem is not based on the treasury percentage. We already allow some tolerance for proportionality, so strictly checking against the treasury ratio isn't strictly necessary.

[CertiK, 03/12/2026]: The team partially mitigated the issue by implementing a delta-based proportional short-dust check in `closeWithLoss()`, which limits excess short-underlying created by the close flow (and fully covers the `DSA == short` path) in commits:

- [1ac5da3a62e13ec0a4796f59ed3c092f156516b2](#)
- [b6ee3aefac56547594850aeca20429495dbde03a](#).

However, it does not fully enforce second-leg DSA proportionality to `closeFractionBps` when `DSA != short`, especially under swap/refund behaviors that leave unused DSA on the position account.

[Venus, 03/13/2026]: The swap is executed in **exact-in mode**, meaning the entire redeemed DSA amount is used as the swap input. As a result, if `dsaAmountToRedeemForSecondSwap` is larger than the amount required to repay `amountToRepaySecond`, the excess does **not remain as unused DSA underlying**. Instead, it results in additional **short tokens** being returned as swap output.

This scenario is already handled by the existing **delta-based** `validateShortDust()` check, which detects and validates any surplus short token balance. Therefore, any excess arising from an oversized `dsaAmountToRedeemForSecondSwap` is properly accounted for and cannot remain untracked.

[CertiK, 03/13/2026]: Under the assumption that the swap operates using an exact-in mechanism, as clarified by the team, the issue described above would not occur. Therefore, the finding is considered resolved in commit [1ac5da3a62e13ec0a4796f59ed3c092f156516b2](#).

VRP-08 | Conservative Rounding Is Not Applied In `_getUtilizationInfo()`

Category	Severity	Location	Status
Logical Issue	● Minor	RelativePositionManager.sol (Base): 1080~1104	● Resolved

Description

The function `_getUtilizationInfo()` utilizes floor division when calculating `nominalCapitalUtilized`, `actualCapitalUtilized`, and `withdrawableAmount`. As a result, the computed utilization values are rounded down, which may slightly overstate the available capital and the amount deemed withdrawable. Rounding in this manner does not follow a conservative approach for risk management, as it can lead to reporting marginally higher available resources than actually exist.

Recommendation

We recommend revising the rounding logic in the `_getUtilizationInfo()` function to ensure all utilization values are rounded up, and all withdrawable amounts are rounded down.

Alleviation

[CertiK, 03/03/2026]: The client made the recommended changes resolving this finding in commit [445a8fb35dbcb9411272d16cde6e23bc2d08fd4b](#).

VRP-09 `getPositionAccountAddress()` Can Return Incorrect Address After Implementation Upgrade

Category	Severity	Location	Status
Logical Issue	● Minor	RelativePositionManager.sol (Base): 708~719	● Resolved

Description

`getPositionAccountAddress()` computes a deterministic clone address using the current `POSITION_ACCOUNT_IMPLEMENTATION`. If the implementation is upgraded, the predicted address changes even for users who already have an existing position account. This results in `getPositionAccountAddress()` returning the wrong address for position accounts created prior to the upgrade.

Recommendation

We recommend updating the code to return the correct address if an account is already deployed and using a previous implementation.

Alleviation

[CertiK, 03/03/2026]: The client clarified that the position account implementation is not intended to be changed and made changes resolving this finding in commit [cee9803e0dc2e349e5d6f7e0b9faea617bf8f2f6](#).

VRP-10 `effectiveLeverage` Is Fixed At Activation And Can Exceed New Max After Collateral Factor Updates

Category	Severity	Location	Status
Logical Issue	● Minor	RelativePositionManager.sol (Base): 292~294	● Resolved

Description

`effectiveLeverage` is validated only at activation using current collateral factors (CFs). If CFs are later reduced, existing positions can have an `effectiveLeverage` that exceeds the new maximum.

Recommendation

We recommend revalidating/reconciling `effectiveLeverage` during `openPosition`.

Alleviation

[CertiK, 03/06/20206]: The client made the recommended changes resolving this finding in commits

- [7d72d378fcf6be2af82cba1e2dd44c9c24448308](#);
- [a1340bf4e8feefd7481216b13e20b37e2f032a06](#).

VRP-01 | Discussion On POSITION_ACCOUNT_IMPLEMENTATION

Category	Severity	Location	Status
Design Issue	● Informational	RelativePositionManager.sol (Base): 57	● Resolved

Description

The variable `POSITION_ACCOUNT_IMPLEMENTATION` is in all caps, although it can be updated via governance.

We would like to understand the design choice to make the `POSITION_ACCOUNT_IMPLEMENTATION` mutable, as this can result in the same user having different position account implementations for different positions, increasing complexity.

If there are expected changes to the implementation that will not require adapting the RPM contract, then we would recommend considering a beacon proxy pattern that would allow all previous position accounts to be upgraded to reference the new implementation as well. For example, if the implementation is upgraded with some gas optimizations, currently any user's current positions will not benefit from them, and a user would need to use a new account to open the position to receive the same benefits.

If expected changes will require adapting the RPM contract, then we recommend making this variable immutable and making necessary changes to the RPM contract for the different position account versions at that time.

Recommendation

We recommend answering the discussion above.

Alleviation

[Venus, 03/03/2026]: The `POSITION_ACCOUNT_IMPLEMENTATION` address is not intended to be modified after the initial deployment. The setter was introduced solely to resolve the circular dependency between the Position Account (PA) and the RelativePositionManager (RPM): The PA requires the RPM address during initialization. The RPM deploys PAs via deterministic cloning. To break this dependency cleanly, we allow the RPM address to be set once on the PA implementation contract. This avoids having to initialize or configure the RPM address individually on each deployed proxy, thereby reducing deployment gas costs. A one-time lock has been added to the setter to ensure it cannot be modified after being set. We do not expect the implementation address to change going forward. Any future functionality can be handled through generic call routing or wrapper logic within the RPM, eliminating the need to upgrade the PA contract itself.

[CertiK, 03/03/2026]: The client resolved this issue by ensuring that `setPositionAccountImplementation()` can only be called once in commit [cee9803e0dc2e349e5d6f7e0b9faea617bf8f2f6](#).

VRP-02 | Discussion On Dust Transfer Reverts For Tokens With Blacklists

Category	Severity	Location	Status
Design Issue	● Informational	PositionAccount.sol (Base): 256~260; RelativePositionManager.sol (Base): 914~916	● Resolved

Description

The dust-sweep step (`_transferDustFromAccountToUser()`) can revert if a token includes a blacklist and the transfer fails due to the `owner` being blacklisted. We would like to clarify that this is the intended behavior and ensure that the protocol's policy is to avoid any interaction with blacklisted addresses. In that case, a revert acts as a hard block on further operations for that account, while liquidation remains possible and protocol solvency is unaffected.

Recommendation

Can you please share the protocols intended policy and procedure for when a position account owner is blacklisted?

Alleviation

[Venus, 03/03/2026]: Issue acknowledged. I won't make any changes for the current version.

This is the intended behavior, if a token transfer reverts due to blacklist restrictions, the protocol will not attempt to bypass it. The account becomes non-operational for user actions, while liquidation remains possible and protocol solvency is unaffected.

VRP-03 | Discussion On Hardcoded 2% Full-Close Repay Bump

Category	Severity	Location	Status
Design Issue	● Informational	RelativePositionManager.sol (Base): 1180~1182, 1237~1239	● Acknowledged

Description

On full closes, the code applies a hardcoded 2% tolerance bump to the repay amount:

`PROPORTIONAL_CLOSE_TOLERANCE = 2` is used in `_validateProfitClose()` and `_validateLossClose()` to increase the repay amount when `closeFractionBps == 100`. This bump flows into `LeverageStrategiesManager.exitLeverage/exitSingleAssetLeverage()` via `borrowedAmountToFlashLoan`. In `_handleExitCollateral()`, debt repayment is capped at `min(flashLoanAmount, actualDebt)`, so the extra amount is not used to repay debt but still incurs higher flashloan fees. In `_handleExitSingleAsset()`, `redeemAmount` is derived from `flashLoanRepayAmount`, so the bump can increase redemption and treasury fee costs.

The 2% bump is imposed by the protocol, but the interface does not expose it or explain it. Users can't opt in, opt out, or choose their own buffer to cover interest accrual during the flash loan. As a result, the system may borrow more than the actual debt, which increases flashloan fees (and in single-asset exits, redemption/treasury costs). This makes quotes harder to predict and removes cost control from users and integrators.

Can you please clarify this design choice and why the alternative solution of utilizing explicit user-provided parameters and documented requirements for the user inputs on full closure was not used?

Recommendation

We recommend clarifying the design choice mentioned above.

Alleviation

[Venus, 03/03/2026]: Issue acknowledged. I won't make any changes for the current version.

This design was chosen to minimize the number of parameters required from the user. Since the close amounts are already computed internally using BPS, the tolerance buffer is applied on-chain to simplify the interface and avoid requiring users to supply additional arguments.

The 2% tolerance is a conservative default intended to account for interest accrual and minor execution variances. It is not a strict requirement, but rather a safety margin to prevent unnecessary reverts. Additionally, this buffer does not materially increase FlashLoan fees.

The fee is calculated on the borrowed amount, and even with a 2% buffer, the incremental fee impact remains negligible. For example, with a typical ~0.1%-0.3% flash loan fee, the extra cost from the 2% tolerance is effectively dust-level.

We acknowledge the finding and confirm that the tolerance was intentionally introduced as a conservative execution safeguard.

VRP-04 | Discussion On Proportional Close Tolerance

Category	Severity	Location	Status
Logical Issue	● Informational	RelativePositionManager.sol (Base): 1174~1176	● Resolved

Description

`PROPORTIONAL_CLOSE_TOLERANCE` is hardcoded at 2% and affects two different policies:

- Entry max leverage via the “friction” term in `_getMaxLeverage()`, making max leverage more conservative.
- Exit proportionality, allowing long redemption amounts up to +/-2% and bumping full-close repay by +2%, which can increase remaining leverage after partial closes.

There is no explicit post-close max-leverage check, so that safety relies on the Comptroller shortfall check.

If market conditions change, the protocol cannot update this value without redeploying the contract. In addition, this value cannot be adjusted per market. Can you please clarify the design decision for keeping this tolerance non-configurable?

Recommendation

We recommend clarifying the design decisions mentioned above.

Alleviation

[CertiK, 03/03/2026]: The client made the recommended changes resolving this finding in commits

- [218b1c6ab6556c4b09b10fb5c0f26b94a7bc412e;](#)
- [8da4d7c861856c6b19d21c82a13eb96a4c905dba.](#)

VRP-11 | Incompatibility With Deflationary Tokens (Non-Standard ERC20 Token)

Category	Severity	Location	Status
Volatile Code	● Informational	RelativePositionManager.sol (Base): 1006	● Resolved

Description

The project design may not be compatible with non-standard ERC20 tokens, such as deflationary tokens or rebase tokens.

The functions use `transferFrom()` / `transfer()` to move funds from the sender to the recipient but fail to verify if the received token amount matches the transferred amount. This could pose an issue with fee-on-transfer tokens, where the post-transfer balance might be less than anticipated, leading to balance inconsistencies. There might be subsequent checks for a second transfer, but an attacker might exploit leftover funds (such as those accidentally sent by another user) to gain unjustified credit.

Scenario

When transferring deflationary ERC20 tokens, the input amount may not equal the received amount due to the charged transaction fee. For example, if a user sends 100 deflationary tokens (with a 10% transaction fee), only 90 tokens actually arrive to the contract. However, a failure to discount such fees may allow the same user to withdraw 100 tokens from the contract, which causes the contract to lose 10 tokens in such a transaction.

Recommendation

We recommend ensuring that no deflationary tokens are supported in the protocol. In addition, we recommend testing any non-standard ERC20 tokens prior to supporting them to ensure they are compatible with all parts of the Venus protocol.

Alleviation

[Venus, 03/03/2026]: Issue acknowledged. I won't make any changes for the current version.

The protocol is not intended to support deflationary, fee-on-transfer, or other non-standard ERC20 tokens.

VRP-12 | Typos And Inconsistencies

Category	Severity	Location	Status
Inconsistency	● Informational	RelativePositionManager.sol (Base): 47~48, 852, 1091	● Resolved

Description

- The comment above `PROPORTIONAL_CLOSE_TOLERANCE` states `100 = 1% margin of error`, however, currently `1=1% margin or error`.
- The comment in `_getUtilizationInfo()` describing the `actualCapitalUtilized` formula is missing a parenthesis.
- The comments above `_redeemLongAndSwapToDSA()` reference `_realizeProfitFromExcessLong` which is not in the codebase.

Recommendation

We recommend fixing the typos and inconsistencies mentioned above.

Alleviation

[CertiK, 03/06/2026]: The client made the recommend changes resolving this finding in commits

- [b32ee9d7925136b8bd3ebb8c483178ab630bfef7](#);
- [29f35a0de3e29315ff0bc167e97f05972215ab91](#).

VRP-13 | Dust Amount Supplied Can Mint 0 VToken

Category	Severity	Location	Status
Logical Issue	● Informational	RelativePositionManager.sol (Base): 896~897, 1008~1009	● Resolved

Description

The `mintBehalf()` function calculates the amount of vToken to mint by dividing the supplied principal amount by the current exchange rate. When the input amount is less than the exchange rate, the division rounds down to zero, resulting in no vToken being minted. The contract does not perform a check or revert in this situation, so the user's supplied amount is accepted but not credited with any vToken, effectively forfeiting this dust amount to the protocol.

Recommendation

We recommend ensuring users are aware of this and do not supply dust amounts, or alternatively, add a check to ensure they are minted at least 1 vToken.

Alleviation

[CertiK, 03/03/2026]: The client made the recommended changes resolving this finding in commit [3a0e2e8a3846e08cfb29ea4469d7fe1f875bb952](#).

VRP-14 | DSA Market Is Not Exited When Deactivated

Category	Severity	Location	Status
Logical Issue	● Informational	RelativePositionManager.sol (Base): 679~696	● Resolved

Description

When a position is deactivated, the contract does not exit the associated DSA market. As a result, the deactivated DSA market remains included in the position's liquidity calculations. If the position is later reactivated with a different DSA market, the new market is entered without exiting the previous one. Over time, this can lead to a position account being entered into multiple DSA markets, causing unnecessary iteration over all markets when computing collateral value. Additionally, the system continues to consider all entered DSA markets, including inactive ones, as collateral, which could lead to unexpected results in future operations.

Recommendation

We recommend ensuring that a DSA market is properly exited whenever a position is deactivated, unless the DSA corresponds to the position's long asset.

Alleviation

[Certik, 03/03/2026]: The client made the recommended changes resolving this finding in commit [c30482037c60d1cf481de7b1aa16bbd6cc2dd967](#).

VRP-15 Potential Incorrect `amountRedeemed` Emitted In `PositionClosed` Event During `closeWithLoss()` When DSA Equals Short

Category	Severity	Location	Status
Logical Issue	● Informational	RelativePositionManager.sol (Base): 576~578	● Resolved

Description

In the function `closeWithLoss()`, when `DSA == short`, the function repays via `exitSingleAssetLeverage()` and ignores the user-supplied `dsaAmountToRedeemForSecondSwap`. However, it still sets `dsaAmountRedeemed = dsaAmountToRedeemForSecondSwap` and emits that value in `PositionClosed`. This means the event can report any arbitrary number that the caller passes, regardless of how much DSA was actually consumed.

Off-chain systems that track principal burned or redeemed using this event will be wrong and can be intentionally misled by the caller.

Recommendation

We recommend emitting the actual redeemed amount (e.g., compute from burned vTokens via `exchangeRateCurrent()` or by measuring underlying balance deltas). Alternatively, set the field to 0 for the single-asset branch and document it as unused.

Alleviation

[CertiK, 03/03/2026]: The client made the recommended changes resolving this finding in commit [11335817160f81ca3657e93043c300d6c1d08bf2](https://github.com/certik/venus/commit/11335817160f81ca3657e93043c300d6c1d08bf2).

VRP-16 | Missing Input Validation

Category	Severity	Location	Status
Logical Issue	● Informational	RelativePositionManager.sol (Base): 554, 560, 1230	● Resolved

Description

In the function `closeWithLoss()`, it does not check that `shortAmountToRepayForFirstSwap == 0` whenever `longAmountToRedeemForFirstSwap == 0` (or when `expectedLongToWithdraw == 0`). The second leg repay amount is reduced based on `shortAmountToRepayForFirstSwap` even if `longAmountToRedeemForFirstSwap == 0`, which causes the first-leg exit to be skipped. This allows user input for the first leg repay to reduce (or zero) the computed second-leg repay without any actual first leg repayment occurring.

Recommendation

We recommend adding the input validations mentioned above.

Alleviation

[Venus, 03/03/2026]: The client made the recommended changes resolving this finding in commit [b7a31dc6380e3f50ba744dbb97473e108c973b7e](#).

VRP-17 `minAmountOut` Parameters Only Guard Slippage Not Total Repayment Requirements

Category	Severity	Location	Status
Logical Issue	 Informational	RelativePositionManager.sol (Base): 428, 431, 531, 534, 869, 1038, 1161, 1203~1204	 Resolved

Description

All `minAmountOut` parameters are enforced only against swap output amounts. They do not inherently ensure that the user will cover full repayment requirements such as flash-loan fees, treasury fee-adjusted redemption amounts, or full-close tolerance bumps. If users don't include those amounts in their `minAmountOut` expectations, transactions can revert or leave residual debt.

Recommendation

We recommend documenting that all the `minAmountOut` inputs should include all required buffers (fees, full-close bumps, treasury fee adjustments). Additionally, we recommend enforcing minimums on-chain when those amounts can be deterministically derived.

Alleviation

[CertiK, 03/03/2026]: The client made changes resolving this finding in commit [b32ee9d7925136b8bd3ebb8c483178ab630bfef7](#).

VRP-18 | External Transfers Into `PositionAccount` Affect Accounting Semantics

Category	Severity	Location	Status
Logical Issue	● Informational	RelativePositionManager.sol (Base): 1251	● Resolved

Description

`PositionAccount` can receive arbitrary `ERC-20/vToken` transfers. These external transfers are interpreted differently depending on whether `DSA == long` or `DSA != long`, which can lead to accounting drift and non-intuitive principal vs collateral classification.

- When `DSA != long`, `_getSuppliedPrincipalBalance()` treats all DSA balance on the position account as principal, regardless of how it arrived. External DSA transfers therefore increase withdrawable principal.
- When `DSA == long`, principal is tracked via `suppliedPrincipalVTokens`, so that external long/DSA vToken transfers are treated as collateral, not principal.

Internal safeguards (e.g., `_syncSuppliedPrincipal`, clamping on burns, dust sweeps) show the team is aware of balance drift, but the semantics for externally transferred assets are still inconsistent.

Recommendation

We recommend documenting this behavior and keeping it tracked as a design invariant.

Alleviation

[CertiK, 03/03/2026]: The client made the recommended changes resolving this finding in commit [9d09241f28a7422b2982f57d62660e80709f32a0](#).

VRP-20 | Missing Emit Events

Category	Severity	Location	Status
Coding Style	● Informational	PositionAccount.sol (Update1): 265	● Resolved

Description

The function `exitMarket()` does not emit an event, whereas other functions in the contract do.

Recommendation

We recommend emitting an event for this function to be consistent.

Alleviation

[CertiK, 03/06/2026]: The client made the recommended changes resolving this finding in commit [3de3c8e4234f3ba79a2f1600ab08370c1f84f43c](#).

APPENDIX | VENUS RELATIVE POSITION MANAGER

Finding Categories

Categories	Description
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Venus Relative Position Manager Security Assessment | CertiK Assessed on Mar 13th, 2026 | Copyright © CertiK

